המכללה האקדמית תל אביב יפו
בית הספר למדעי המחשב

תכנות מכוון עצמים ושפת ++C, 121119

גב' קרן כליף , מר אמיר קירש
תשע"ט, סמסטר א', מועד א', 22/01/2019

מספר זהות | | | | | | | | |

משך הבחינה: שעתיים וחצי
חומר עזר: כל חומר עזר כתוב מותר, למעט מכשירים אלקטרוניים למיניהם
אין להעביר חומר עזר בין תלמידים במהלך הבחינה

בבחינה 15 שאלות:
10 שאלות חובה (שאלות 1-10), משקל כל שאלה 10 נקודות.
5 שאלות בונוס (שאלות 11-15), משקל כל שאלה 5 נקודות.

בחישוב הניקוד יילקחו בחשבון כל התשובות הנכונות. ציון מקסימלי בבחינה – 100.
בכל שאלה יש לסמן את התשובה הנכונה ביותר (אחת בלבד), באופן ברור, בדף התשובות בלבד. אם סימנת יותר מתשובה אחת - השאלה תיפסל!
יתקבלו תשובות שנכתבו על גבי דף התשובות בלבד. לא ייבדקו תשובות שלא ייכתבו בדף המיועד לכך.

סימון תשובות בטופס הבחינה לא יהווה בסיס לערעור.

יש להחזיר את דף התשובות ביחד עם טופס הבחינה, אחרת הבחינה לא תיבדק.

בהצלחה!

קטע קוד A

```
class A {
    static A* first;
    static A* last;
    A *prev, *next; // 2 pointers to A
public:
    A(): prev(last), next(nullptr) {
        if(last) last->next = this;
        last = this;
        if(!first) first = this;
    }
    A(const A& a) : A() {}
    A& operator=(const A&) = delete;
    virtual ~A() {
        if(prev) prev->next = next;
        if(next) next->prev = prev;
        if(first == this) first = next;
        if(last == this) last = prev;
    }
    friend ostream& operator<<(ostream& out, const A& a) {
        a.printSelf();
        if(a.next) out << *(a.next);
        else out << "END" << endl;
        return out;
    }
    virtual void printSelf()const = 0;
};

class B: public A {
    int *refCounter, *ptr;
public:
    B(int i = 42): refCounter(new int(1)), ptr(new int(i)) {}
    B(const B& b): A(b), refCounter(b.refCounter), ptr(b.ptr) {
        ++(*refCounter);
    }
    ~B() override {
        cout << * this;
        if(--(*refCounter) == 0) {
            delete ptr;
        }
    }
    void printSelf()const override { cout << *ptr << endl; }
};

// in A.cpp
A* A::first = nullptr;
A* A::last = nullptr;
```

להלן מספר main אפשריים:

A1 main

```
int main() {
  B* pb1 = new B(7);
  B b2 = *pb1;
  {
    B b3;
    delete pb1;
  }
}
```

A2 main

```
int main() {
  A* pa = new B(13);
  B b = *pa;
  cout << *pa;
}
```

A3 main

```
int main() {
  A* pa = new B(13);
  B* pb = dynamic_cast<B*>(pa);
  cout << *pb;
}
```

A4 main

```
class C: public A {
  virtual void printSelf()const override {
    static int i = 0; // a reminder: born once and kept alive
    cout << ++i << endl;
  }
};

int main() {
  B b1;
  C c1;
  cout << b1;
  cout << c1;
}
```

קטע קוד B

```
class Pet {
public:
    virtual ~Pet() {}
};

class Dog: public Pet {};

class Cat: public Pet {};

class Godzilla: public Pet {};

class CountByType {
    map<string, int> countByType;
public:
    template<class T>
    void count(const list<T*>& all) {
        for(auto item : all) {
            countByType[typeid(*item).name()]++;
        }
    }
    void print() const {
        // note: order of the loop is known
        // (see comment in Hebrew below the code section)
        for(const auto& pair : countByType) {
            cout << pair.first << ": " << pair.second << endl;
        }
    }
};
```

שימו לב:
המחלקה map ממויינת לפי ה-key והמיון של string הוא לקסיקוגרפי (כלומר כמו מיון של מילים במילון).

כמו כן:
הנח ש- `typeid(type).name()` מחזיר:
- עבור אובייקטים - את שם המחלקה בצירוף המילה `class`.
- עבור טיפוסים בסיסיים פשוט את שם הטיפוס:
  "`int`" עבור `int`, "`char`" עבור `char`, "`int*`" עבור `int*` וכו'.

להלן מספר main אפשריים:

B1 main

```
 1. int main() {
 2.     Dog rexi, rocky;
 3.     Godzilla g1, g2;
 4.     Cat mitzi;
 5.     // add all items inside the {} into the list
 6.     list<Pet*> pets = {&rexi, &rocky, &g1, &g2, &mitzi};
 7.     CountByType counter;
 8.     counter.count(pets);
 9.     counter.print();
10. }
```

שימו לב:
הקוד בשורה 6:

```
list<Pet*> pets = {&rexi, &rocky, &g1, &g2, &mitzi};
```

מייצר רשימה מקושרת של השפה ומכניס אליה את הערכים שנמסרו בתוך המסולסליים.

B2 main

```
 1. int main() {
 2.     CountByType counter;
 3.     int i = 0;
 4.     // add all items inside the {} into the list
 5.     // (a single item in this case)
 6.     list<int*> list1 = {&i};
 7.     counter.count(list1);
 8.     counter.count(list<int*>{&i}); // a temp list with single item
 9.     // add all items inside the {} into the list
10.     // (a single item in this case)
11.     list<const char*> list2 = {"Hello"};
12.     counter.count(list2);
13.     counter.print();
14. }
```

קטע קוד C – שאלות בונוס

```
template<class T>
class Stack {
    T** arr = nullptr;
    int size = 0;
    int pos = 0;
public:
    Stack(int size1):size(size1) { arr = new T*[size]{}; }
    ~Stack() {
        for(int i=0; i<pos; ++i) {
            delete arr[i];
        }
        delete []arr;
    }
    Stack(Stack& other) = delete;
    friend void swap(Stack& first, Stack& second) {
        std::swap(first.arr, second.arr);
        std::swap(first.size, second.size);
        std::swap(first.pos, second.pos);
    }
    Stack& operator=(Stack other) { swap(*this, other); }
    Stack(Stack&& other) { swap(*this, other); }
    Stack& push(T t) {
        arr[pos++] = new T(std::move(t));
        return *this;
    }
    T pop() {
        T* retptr = arr[--pos];
        T retval = std::move(*retptr);
        delete retptr;
        return retval;
    }
    friend ostream& operator<<(ostream& out, const Stack<T>& s) {
        for(int i=0; i<s.pos; ++i) {
            out << *s.arr[i] << ' ';
        }
        return out << endl;
    }
};

Stack<int> createIntStack(int size) {
    Stack<int> s(size);
    for(int i=0; i<size; ++i) {
        s.push(i);
    }
    return s;
}
```

להלן מספר main אפשריים:

C1 main

```
int main() {
    const char *str1 = "hello", *str2 = "world";
    Stack<string> s(10);
    s.push(str1).push(str2);
    cout << s;
    cout << s.pop();
}
```

C2 main

```
int main() {
    Stack<Stack<int>> s(1);
    Stack<int> sint = createIntStack(3);
    s.push(sint);
    cout << s.pop();
    cout << s;
}
```

C3 main

```
int main() {
    Stack<Stack<int>> s(10);
    s.push(createIntStack(3)).push(createIntStack(9));
    cout << s;
    auto sint = s.pop();
    cout << sint.pop() << endl;
    cout << s.pop();
}
```

בהצלחה !